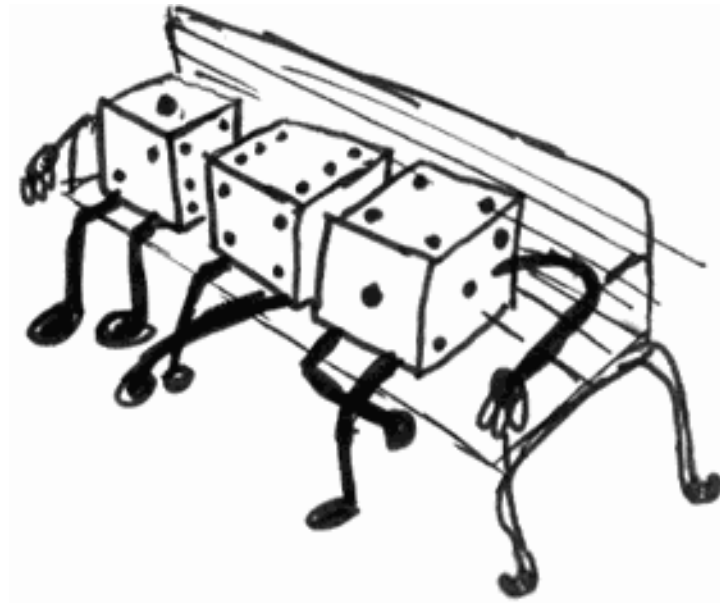


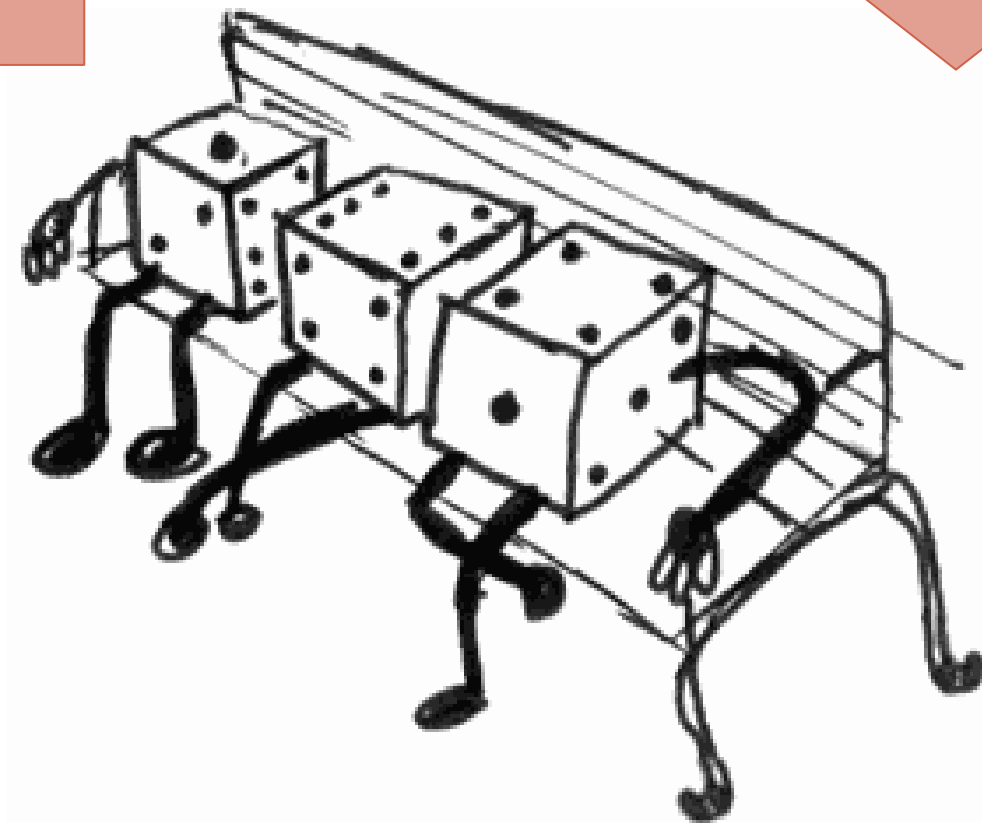
BANCO DE DADOS II



Prof. Michel Junio Ferreira Rosa
Ciência da Computação

Na aula anterior (Aula 06):
Outros objetos

Na aula de hoje (Aula 07):
Indexação



Índices

- Estrutura para melhorar o procedimento de consultas;



Índices

- Objetivo:
 - Reduzir o número de I/O no disco para localizar dados solicitados na consulta;
- Pense em um sumário ou índice remissivo de um livro
 - Quantas páginas você acessaria procurando algo no livro sem o sumário ou índice remissivo??

Índices

- O(s) campo(s) cujos valores o índice se baseia formam a chave de pesquisa;
- Índices sempre são associados a tabelas e a uma ou mais colunas. Uma tabela pode ter vários índices;
- Um índice rastreia os dados de uma ou mais colunas num mecanismo próprio, permitindo as cláusulas WHERE obterem seus valores mais rapidamente;

Índices

- Índices são, portanto, **estruturas de dados auxiliares** cujo único propósito é tornar mais **rápido o acesso** a registros baseado em certos campos, chamados campos de indexação;
- **OBSERVAÇÃO 1:** Normalmente a tabela deve ter mais que 1000 linhas para que o índice seja eficiente;
- **OBSERVAÇÃO 2:** Índices demais é ruim, pois ao usar os comando INSERT, UPDATE ou DELETE, o SGBD atualiza a tabela de índice;

Organização do Índice

- Índice Denso
 - Sequência de blocos contendo apenas as chaves dos registros e os ponteiros para os próprios registros;
 - Índice denso = (chave-ponteiro, registro)

Índice Denso

Curitiba		Curitiba	João
Florianópolis		Curitiba	José
Porto Alegre		Curitiba	Lucas
São Paulo		Curitiba	Maria
Vitória		Florianópolis	Marta
		Florianópolis	Antônio
		Florianópolis	Sebastião
		Porto Alegre	Sônia
		São Paulo	Carla
		São Paulo	Joaquim
		São Paulo	Manoel
		Vitória	Tereza
		Vitória	Jorge

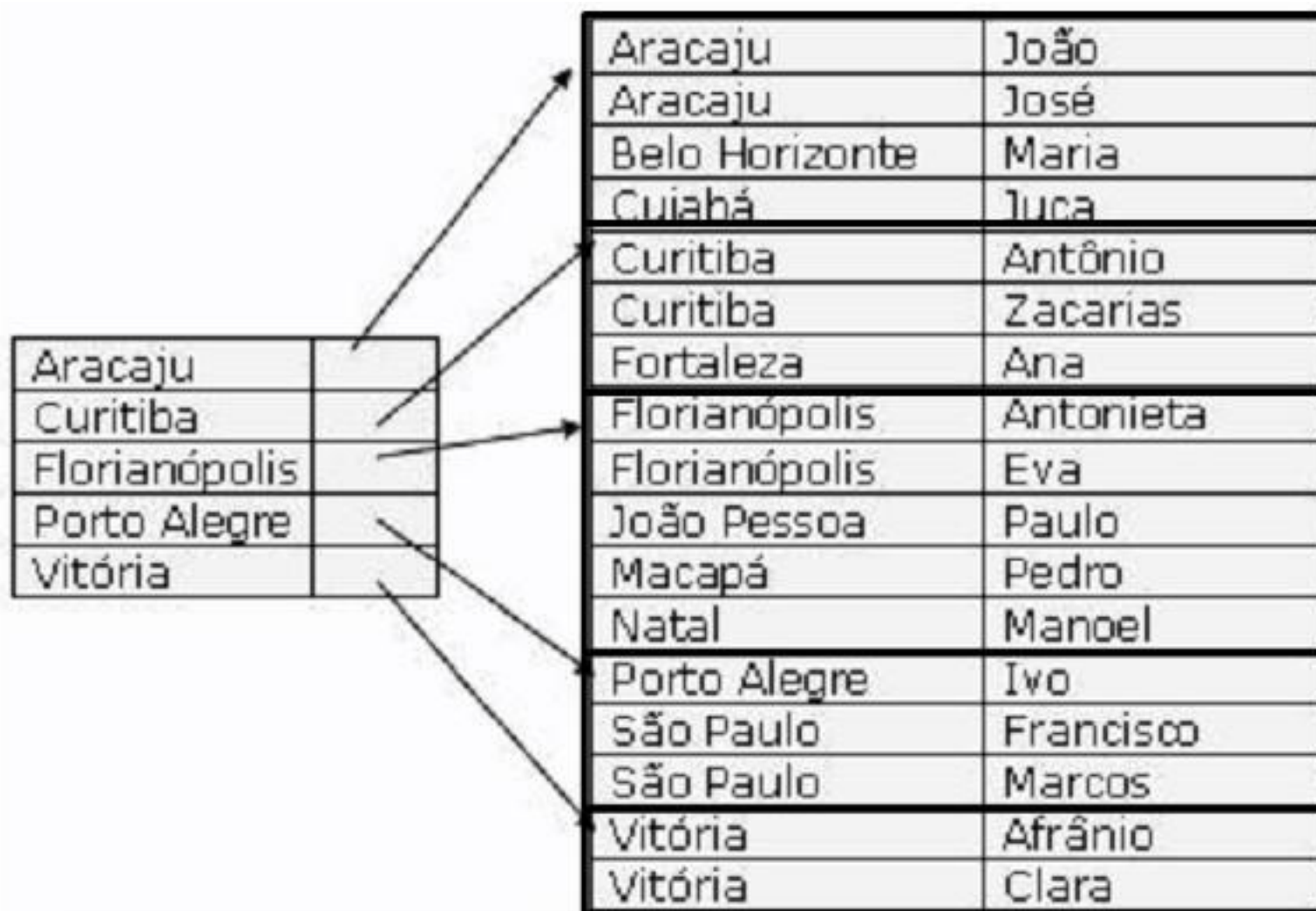
Organização do Índice

- Índice Esparso
 - Usa menos espaço de armazenamento que o índice denso ao custo de um tempo um pouco maior para localizar um registro dada a sua chave;
- Índice esparsos = (chave-ponteiro para bloco, blocos de dados);



Aponta para o 1º registro do bloco

Índice Esparso



Índice Denso X Esparso

- Os índices densos são preferíveis comparados aos índices esparsos, devido a possibilidade de encontrarmos com mais agilidade o desejado;
- A vantagem dos índices esparsos é o fato de ocuparem pouco espaço em disco e menos trabalho na manutenção em inserções e exclusões;

Tipos de Índices

Índices de agrupamentos (clustered)

versus

Índices secundários (nonclustered)

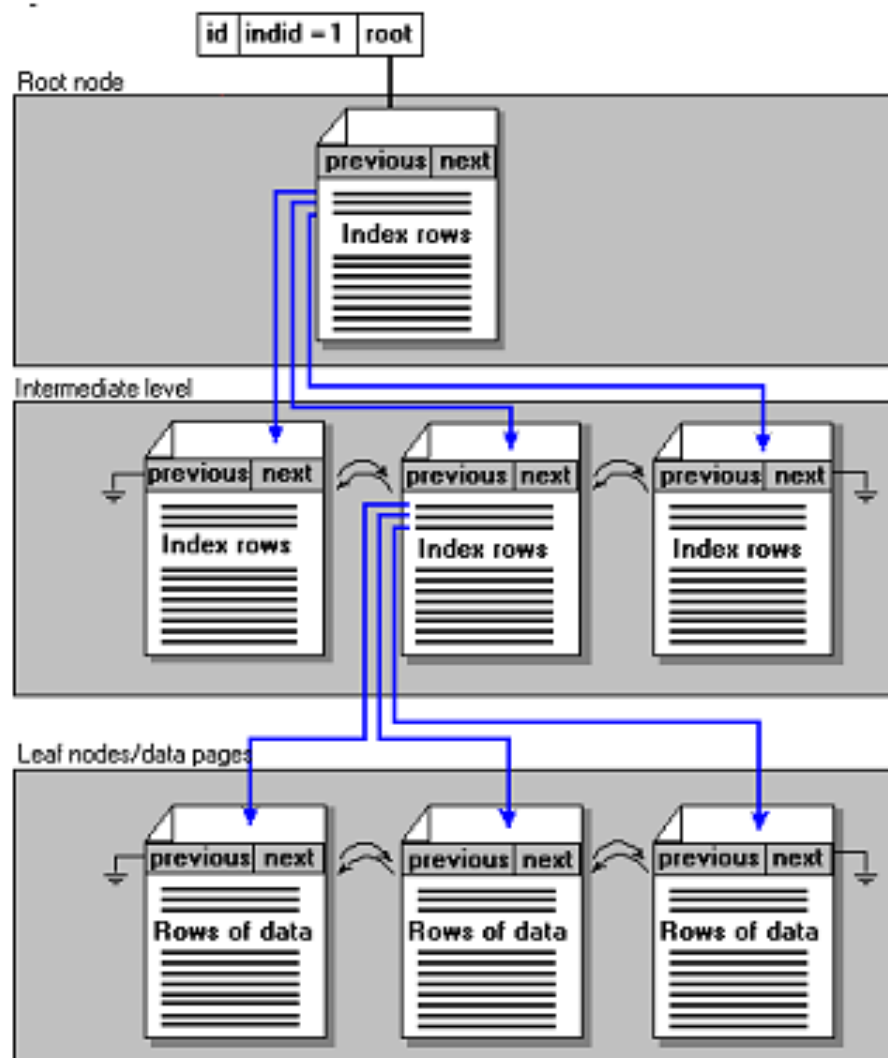
Índices clustered

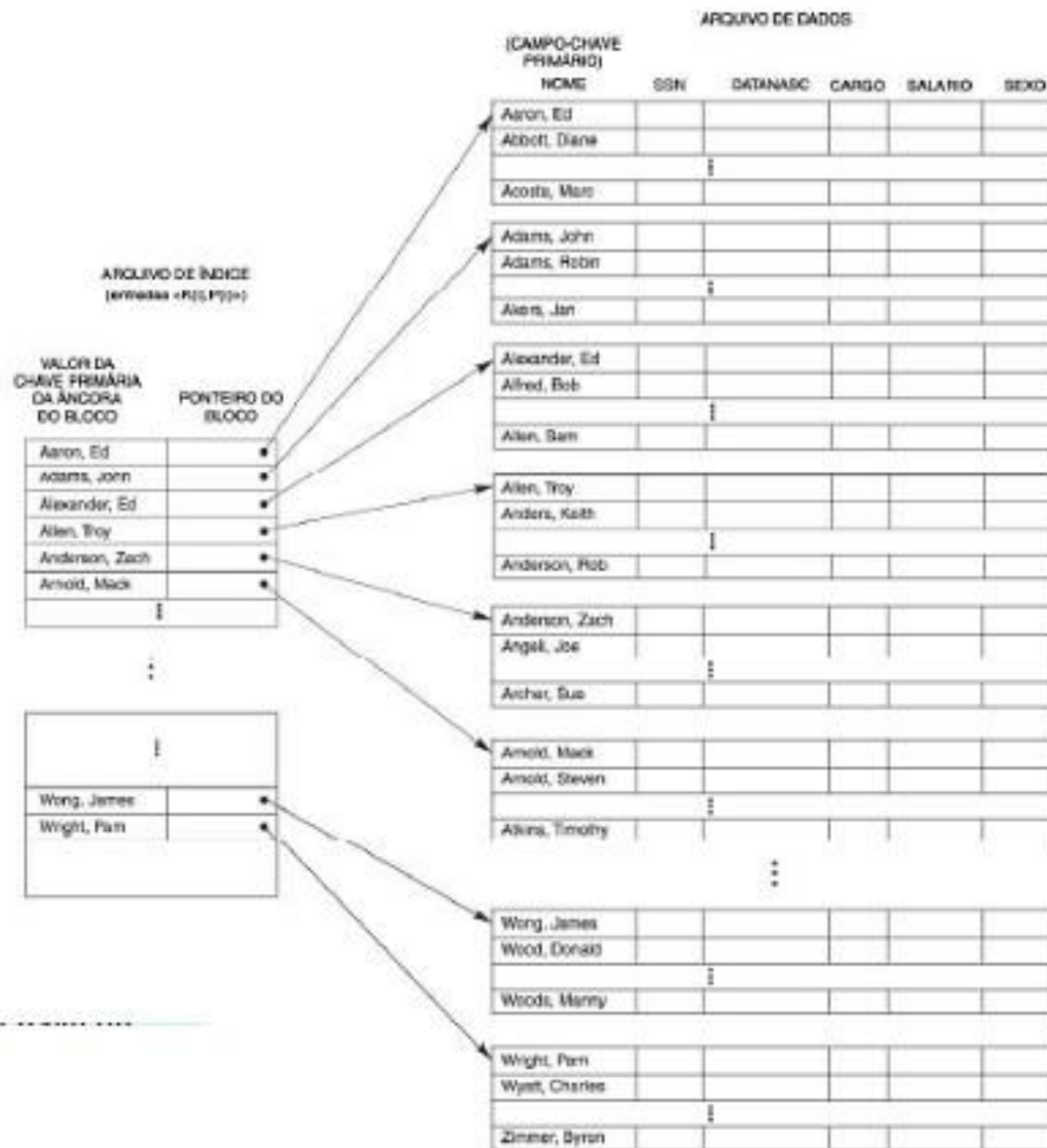
- Ordena os dados(fisicamente) de acordo com os valores do campo onde o índice foi definido;
- Torna as consultas bem mais eficientes, mas cria um *overhead* nas operações de insert, update e delete pela possibilidade de reordenação física dos dados;

Índices clustered

- A implementação de um índice clustered geralmente é feito usando uma B-Tree;
- Dados são divididos em páginas;
- Cada página de um índice possui o cabeçalho que identifica a página mais as linhas do índice;
- Cada linha do índice contém um valor-chave e um ponteiro para outra página do índice ou para um registro de dados;
- Páginas do mesmo nível são ligadas por uma lista duplamente encadeada;

Índice clustered





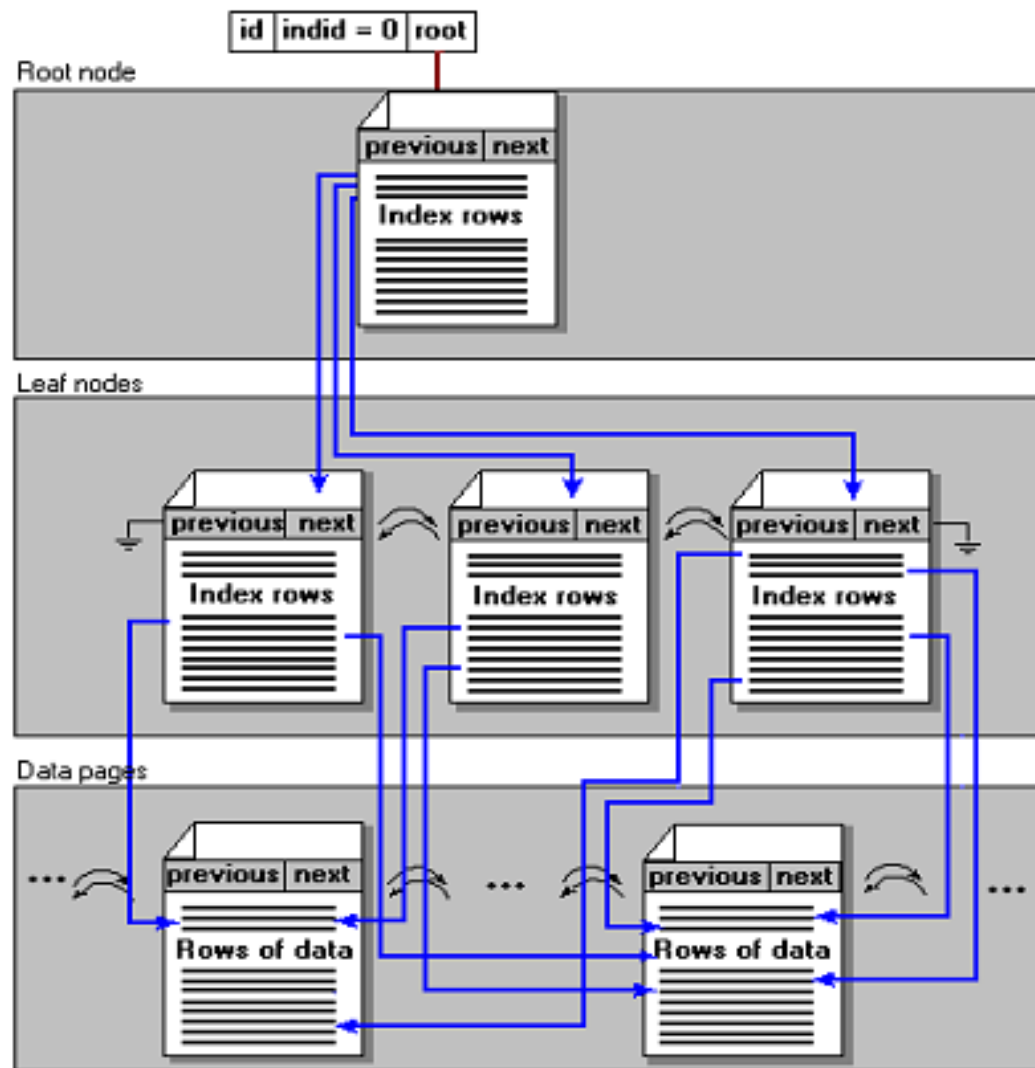
Índice clustered são muito bons para

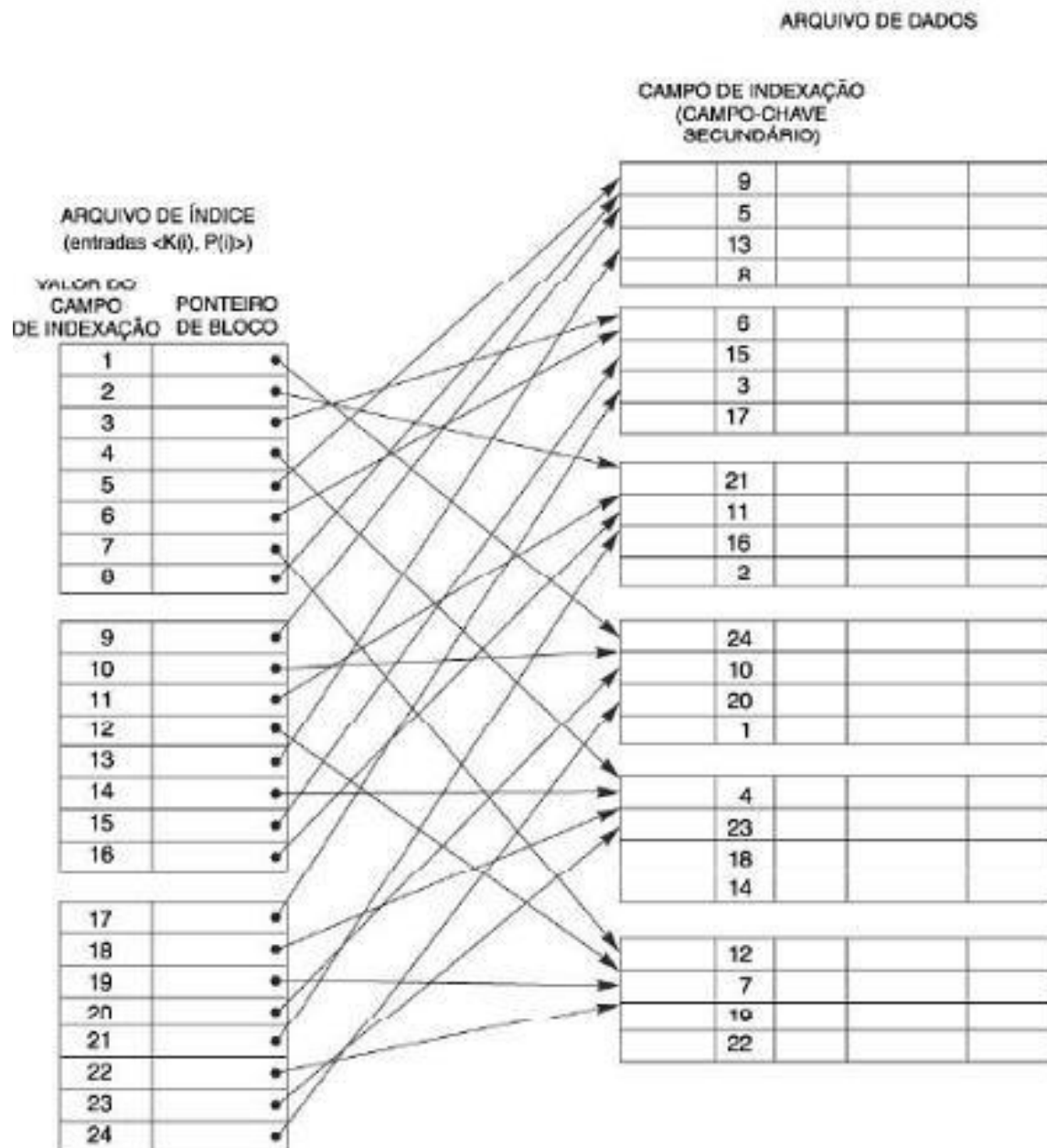
- Colunas que contém grande número de valores distintos;
- Consultas que retornam um range de valores usando operadores como BETWEEN, >, >=, <, and <=;
- Colunas que são acessadas sequencialmente;
- Consultas que retornam grandes conjuntos de resultados;
- Colunas que são acessadas frequentemente por consultas envolvidas em join ou group by;

Índices nonclustered

- Cria uma estrutura separada das páginas da tabela, ou seja, a ordem dos registros da tabela não é alterada;
- O campo de indexação é um campo não ordenado do arquivo de dados. Podem haver muitos índices nonclustered uma mesma página;
- Como não envolve a ordenação física dos dados, podemos ter vários índices nonclustered para uma mesma tabela;

Índices noclustered





Observações sobre índices

- Por padrão quando uma chave primária é definida, automaticamente é criado um índice clustered para a tabela, caso não exista um;
- Um campo com restrição unique também cria automaticamente um índice nonclustered para a tabela.

Fatores de eficiência do uso de índices

- Número de blocos de índices em geral é pequeno quando comparado com o número de blocos de dados;
- Tendo em vista que as chaves são classificadas a pesquisa é rápida (pode-se usar um algoritmo de pesquisa binária);

Fatores de eficiência do uso de índices

- O índice pode ser pequeno o bastante para ser mantido permanentemente em buffers da memória principal;
- Nesse caso, uma pesquisa para uma determinada chave envolve apenas acessos à memória principal, sem precisar de operação de I/O

CONSIDERAÇÕES SOBRE OS ÍNDICES

Benefícios dos índices

- O uso de índices pode ainda ser mais valioso em consultas envolvendo JOINS ou múltiplas tabelas:
 - Consultas em uma tabela o número de valores que precisam ser examinados por coluna é o número de registros da tabela;
- Em consultas em múltiplas tabelas, o número de possíveis combinações aumenta.

Benefícios dos índices

- Suponha que existam três tabelas sem índices, t1, t2 e t3, cada uma contendo as colunas c1, c2 e c3, respectivamente, e cada coluna contenha 1.000 registros. Suponha ainda a consulta abaixo:

```
SELECT  t1.c1, t2.c2, t3.c3  
FROM    t1, t2, t3  
WHERE   t1.c1 = t2.c2 AND t2.c2 = t3.c3;
```

Benefícios dos índices

- O resultado desta consulta deve ser uma tabela de 1.000 registros, cada um contendo três valores iguais;
 - Se a consulta for executada sem o uso de índices, todas tuplas têm que ser percorridas;
- Consequentemente, o banco tenta todas as combinações possíveis para encontrar os registros que combinam com a condição da cláusula WHERE;
- O número de possíveis combinações é $1.000 \times 1.000 \times 1.000 = 1.000.000.000$;

Benefícios dos índices

- O uso de índices nas tabelas melhora o tempo de resposta da consulta, pois permitem que consultas sejam assim processadas:
 - Selecione a primeira tupla da tabela t1 e veja o seu valor;
 - Usando o índice da tabela t2, vá diretamente à tupla que combine com o valor de t1. Ainda, use o índice da tabela t2 para ir diretamente à tupla que combine com o valor de t3;
 - Vá para o próximo registro da tabela t1 e repita o procedimento anterior. Faça isto até que todas as linhas em t1 tenham sido examinadas.

Custo do índices

- Há custos de espaço e tempo. Na prática, as desvantagens tendem a serem minimizadas pelo valor das vantagens;
- Índices aumentam a velocidade de consultas, mas tornam mais lentas as operações de escrita (inserções, atualizações e remoções), pois gravar uma tupla requer não apenas escrevê-la, mas também mudar a ordenação dos índices;

Custo dos índices

- Quanto mais índices houver em uma tabela, mais mudanças de ordenação podem ser necessárias, o que pode comprometer o desempenho;

Material de Apoio Externo

- Simulação B-Tree:
 - <https://www.cs.usfca.edu/~galles/visualization/BTree.html>
- Documentação PostgreSQL:
 - <https://www.postgresql.org/docs/current/indexes-types.html#INDEXES-TYPES>

Escolha dos índices

- Índices devem ser criados em colunas usadas para pesquisa, ordenação ou agrupamento, ou seja, nas que aparecem na cláusula WHERE, em JOINS, ORDER BY ou GROUP BY ;
- Colunas que aparecem em cláusulas JOIN ou em expressões como `col1 = col2` na cláusula WHERE são fortes candidatas à criação de um índice;

Escolha dos índices

- É importante utilizar índices em chaves estrangeiras, já que estas são muito utilizados em JOINS;

Dicas Práticas

- Índices funcionam melhor em colunas que contenham um alto número de valores distintos;
 - Mantenha, sempre que possível, chaves primárias pequenas (os SGBDs criam um índice para cada chave primária);
- Não crie índices em excesso. Índices devem ser atualizados e reorganizados quando o conteúdo de uma tabela é modificado. Logo, quanto mais índices, mais demorada será a atualização;

Dicas Práticas

- Crie índices com valores pequenos;
 - Use tipos de dados o menor possível;
- Por exemplo, não use uma coluna BIGINT se SMALLINT suporta os dados que serão armazenados. Não use VARCHAR(100) se nenhum dos valores armazenados ultrapassa 25 caracteres.

Dicas Práticas

- Valores pequenos melhoram o processamento de índices de muitas maneiras:
 - Podem ser comparados mais rapidamente;
 - Ocupam menos espaço de disco nos arquivos de índices;
 - É possível a permanência de mais registros em cache, fazendo com que o servidor tenha menos acessos a disco.

Dicas Práticas

- Se um índice é raramente ou nunca utilizado, este diminui, desnecessariamente, o desempenho da tabela;
- As expressões de índice devem ser utilizadas somente quando as consultas que usam o índice forem muito frequentes;

ÍNDICES EM SQL

Índice em SQL

- Sintaxe:

```
CREATE INDEX <nome_indice> ON  
<nome tabela> (<lista atributos>)
```

Exemplo:

```
CREATE INDEX meu_indice meu_indice ON  
agencia(nome_agencia)
```

Índices em SQL

- Excluir um índice

DROP INDEX <nome_indice>

- Exemplo

DROP INDEX meu_indice

ÍNDICES NOS SGBDS

PostgreSQL

- **B-tree**: implementa árvores B e é o método padrão.
 - Usado nas seguintes operações: <, <=, =, >=, >, LIKE, BETWEEN;
- **Hash**: Cria uma tabela de hashing para indexação.
 - Somente usado nas operações de igualdade (=). Seu uso não é recomendado;
- **R-tree**: deve ser usado somente para indexar tipos de dados geométricos (box, line, circle, etc);
- **Gist** (Generalized Index Search Trees): fornece uma estrutura de árvore padrão para ser estendidas e pode ser usada com novos tipos de dados;

MySQL

- Todos os índices do MySQL (PRIMARY, UNIQUE e INDEX) são armazenados em árvores B;
- Um índice é usado para colunas que você compara com os seguintes operadores: =, >, >=, <, <=, BETWEEN ou um LIKE com um padrão que começa com um prefixo sem meta caracteres como 'algo%'.

SQL Server

- Índices no SQL Server são construídos sobre estruturas B-Tree;

OBS:

- A sintaxe de criação e exclusão de índices nos SGBDs é a mesmo do SQL.

Considerações Finais

- Indexação é uma ferramenta importante para o aumento de desempenho das consultas;
- Use indexação como a primeira alternativa para obter um ganho de desempenho e depois avalie que outras técnicas podem ser úteis;
- Quando uma consulta demora a ser concluída, normalmente as tabelas envolvidas não possuem índices ou estes foram mal criados. A adequação ou criação dos índices necessários resolve o problema na grande maioria das vezes.

Dúvidas??????????



Referências

- ELMASRI, R; NAVATHE, S. B. **Sistemas de Bancos de Dados**. São Paulo. Addison Wesley, 4ed, 2005.
- RAMAKRISHNAN, R. **Sistema de Banco de Dados**, McGraw-Hill, 1999

